



Java Network Monitor

Sistema distribuído de monitoramento de máquinas em rede

Trabalho de Redes • PUC Minas • 2026

Felipe Barros • Laura Menezes • Mateus Ribeiro • Matheus Henrique • Vitor Gomes

Agenda

01 Contexto e requisitos

02 Visão geral da solução

03 Arquitetura e protocolo

04 Interface e concorrência

05 Testes: Packet Tracer e Wireshark

06 Conclusão e próximos passos

Objetivo e Requisitos



Interface

Java Swing

Painel gráfico para
visualização em tempo real



Protocolos

TCP + UDP

TCP para registro,
UDP para heartbeat



Concorrência

Multithreading

Threads paralelas para
monitorar múltiplas máquinas



Topologia

Redes distintas

Comunicação entre
redes IP distintas



Simulação

Cisco Packet Tracer

Validação de rotas
e conectividade



Análise

Wireshark

Captura e inspeção
de pacotes na rede

Solução Proposta



Visão Geral

Arquitetura cliente-servidor em Java

- Cliente Swing + servidor Java
- Monitoramento de CPU, RAM e Disco em tempo real
- TCP para registro e consultas (porta 8080)
- UDP para status e heartbeat (porta 9090)



Protocolo confiável

TCP

8080

Garante entrega ordenada de dados para registro de máquinas e consultas ao servidor na porta 8080.



Baixo overhead

UDP

9090

Envio periódico de métricas (CPU, RAM, disco) com mínimo impacto na rede via porta 9090.

Arquitetura do Sistema



CLIENTE

Interface Swing + comunicação

- ConnectionFrame — Tela de conexão
- MainFrame — Dashboard principal
- MetricsSimulator — Simula métricas
- TcpClient — Comunicação TCP
- UdpClient — Envio de status

TCP

8080



SERVIDOR

Gerenciamento e monitoramento

- TcpServer — Aceita conexões
- UdpServer — Recebe métricas
- ClientHandler — Gerencia clientes
- MonitoringService — Lógica central
- ConnectedMachine — Modelo de dados

UDP

9090

TCP



UDP

Protocolo de Comunicação

TCP — porta 8080

REGISTER	Registro do cliente
REGISTER_OK	Confirmação de registro
GET_MACHINE_LIST	Solicita lista de máquinas
MACHINE_LIST	Resposta com métricas
GET_MACHINE_DETAILS	Detalhes de uma máquina
DISCONNECT	Encerramento de conexão

UDP — porta 9090

HEARTBEAT	Sinal de vida (a cada 5s)
STATUS	Métricas detalhadas (a cada 2s)

Por que UDP para métricas?

- Baixo overhead e latência mínima
- Perda eventual de pacote é aceitável
- Envio frequente garante dados atualizados

Interface e Multithreading

Dashboard Swing

Interface gráfica em tempo real

- 1 Barras de progresso para CPU e RAM
- 2 Gráfico histórico de métricas
- 3 Tabela de máquinas conectadas
- 4 Atualização automática em tempo real

Swing

Real time

Threads Principais

- 1 **TcpServer**
Aceita conexões, cria ClientHandler por cliente
- 2 **UdpServer**
Loop de recepção de datagramas em paralelo
- 3 **ClientHandler**
Thread dedicada por cliente TCP
- 4 **SwingWorker**
Atualização segura da UI fora da EDT

Threads

Paralelo

Topologia e Simulação



Rede R1

192.168.0.0/16

PC0 — Cliente



Rede R2

172.16.0.0/12

Interligação



Rede R3

10.0.0.0/8

PC1 — Servidor

Roteadores

3× Cisco 2911

Roteamento

Estático (ip route)

Simulador

Cisco Packet Tracer

Análise

Wireshark



Conclusão e Próximos Passos

✓ Requisitos Atendidos

- Swing + interface gráfica
- TCP (registro e consultas)
- UDP (status e heartbeat)
- Multithreading
- Cisco Packet Tracer
- Análise com Wireshark

Aprendizados

- Diferenças práticas TCP vs UDP
- Desafios de port forwarding e roteamento
- Concorrência e atualização segura da UI

Próximos Passos

- Persistência de métricas (banco de dados)
- Autenticação dos clientes
- Dashboard web (React)